
Minimizing Weighted Completion Times with Deadlines

Author(s): Marc E. Posner

Source: *Operations Research*, Vol. 33, No. 3 (May - Jun., 1985), pp. 562-574

Published by: INFORMS

Stable URL: <http://www.jstor.org/stable/170558>

Accessed: 17/03/2009 06:29

Your use of the JSTOR archive indicates your acceptance of JSTOR's Terms and Conditions of Use, available at <http://www.jstor.org/page/info/about/policies/terms.jsp>. JSTOR's Terms and Conditions of Use provides, in part, that unless you have obtained prior permission, you may not download an entire issue of a journal or multiple copies of articles, and you may use content in the JSTOR archive only for your personal, non-commercial use.

Please contact the publisher regarding any further use of this work. Publisher contact information may be obtained at <http://www.jstor.org/action/showPublisher?publisherCode=informs>.

Each copy of any part of a JSTOR transmission must contain the same copyright notice that appears on the screen or printed page of such transmission.

JSTOR is a not-for-profit organization founded in 1995 to build trusted digital archives for scholarship. We work with the scholarly community to preserve their work and the materials they rely upon, and to build a common research platform that promotes the discovery and use of these resources. For more information about JSTOR, please contact support@jstor.org.



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*.

Minimizing Weighted Completion Times with Deadlines

MARC E. POSNER

New York University, New York, New York

(Received May 1983; accepted June 1984)

We examine a one-machine scheduling problem whose objective is to minimize the weighted sum of completion times, subject to deadline constraints. We synthesize and extend current known theory, and derive precedence relations between jobs. These relationships, along with tight new bounding criteria, are incorporated in a branch and bound solution procedure.

CONSIDER the processing of n jobs on one machine. Suppose that each job has a weight related to the urgency of completion, as well as a deadline. Our problem is to find an ordering of these jobs that minimizes the sum of weighted completion times, subject to meeting all deadlines.

This problem was first studied by Smith [1956], who found a simple solution procedure both for situations with no deadlines, and for situations with deadlines, but with equal completion time weights. Emmons [1975] and Burns [1976] showed that Smith's procedure does not extend to the case of unequal weights. Burns constructed a pairwise interchange heuristic for the problem, and Miyazaki [1981] presented an improved heuristic. Bansal [1980] developed a procedure to find an optimal solution, using a branch and bound algorithm and dominance criteria, and employing one of Smith's methods to calculate a lower bound. This optimal procedure was improved by Potts and Van Wassenhove [1983], who found additional dominance criteria and a computationally superior bound. Shanthikumar and Buzacott [1982] suggested a decomposition approach to solve the problem.

Rinnooy Kan [1976] and Hariri and Potts [1983] discuss the closely related problem of minimizing the sum of weighted completion times subject to release dates. Despite appearances, this problem is not equivalent to the one we have described, because the constraint direction permits some results with deadlines that are not possible with restricted arrivals. Other work, including that by Ershler et al. [1976, 1983],

Subject classification: 581 one-machine scheduling problem, 584 weighted completion times with deadlines.

examines the problem of simultaneously satisfying release dates and deadlines.

We begin by synthesizing and extending the known theory concerning this problem. We then develop new precedence relationships which, along with a new lower bound criterion, are used in a branch and bound algorithm. We show our bound to be theoretically superior to the bounds found by Bansal and by Potts and Van Wassenhove. Computational efficiency is also discussed.

1. NOTATION AND PRELIMINARIES

Let job i for $i = 1, \dots, n$ have processing time p_i , weight w_i , and deadline d_i where p_i, w_i, d_i are positive integers. Let S be the set of all $n!$ orderings of the n jobs. If $\sigma \in S$ is a particular ordering, then define $\sigma(i)$ as the job that appears in position i of the ordering σ .

The problem can be formally stated as follows:

$$(P1) \quad \begin{aligned} &\text{minimize}_{\sigma \in S} Z(\sigma) = \sum_{i=1}^n w_{\sigma(i)} \sum_{j=1}^i p_{\sigma(j)} \\ &\text{subject to} \quad \sum_{j=1}^k p_{\sigma(j)} \leq d_{\sigma(k)}, \quad k = 1, \dots, n. \end{aligned}$$

Let σ^* be an optimal ordering for problem (P1). If job i is to precede job j in a particular ordering, we write $i < j$.

2. BASIC RESULTS

This section studies the theoretical structure of (P1). The first proposition is due to Smith, while Corollary 1.1 can be found in Potts and Van Wassenhove.

PROPOSITION 1. *If $\sigma = (1, 2, \dots, n)$ is not feasible and $d_1 \leq d_2 \leq \dots \leq d_n$, then problem (P1) has no feasible solution.*

Proof. By assumption, $\sum_{k=1}^r p_k > d_r$ for some $r \leq n$. Consider an arbitrary sequence σ' and let $t = \max\{i \mid 1 \leq \sigma'(i) \leq r\}$. Since $\{1, 2, \dots, r\} \subseteq \{\sigma'(1), \sigma'(2), \dots, \sigma'(t)\}$, $\sum_{i=1}^t p_{\sigma'(i)} \geq \sum_{k=1}^r p_k > d_r \geq d_{\sigma'(t)}$ and σ' is not feasible.

As a result of Proposition 1, we assume throughout that $d_1 \leq d_2 \leq \dots \leq d_n$ and that $\sigma = (1, 2, \dots, n)$ is feasible.

COROLLARY 1.1. *If $d_r < p_t + \sum_{k=1}^r p_k$ for some r and t where $r < t$, then $i < t$ for $i = 1, 2, \dots, r$.*

Proof. By Proposition 1, every sequence with $\sigma(i) = t$ and $i \leq r$ is infeasible.

COROLLARY 1.2. *If $\sum_{k=1}^n p_k > d_{n-1}$, then $\sigma(n) = n$.*

Proposition 2 is a unifying result. It is a modification of the result found in Burns and gives a necessary and sufficient criterion to determine the relative costs of interchanging two jobs. Precedence relations that can be established from these relative costs will be used effectively in our branch and bound algorithm. Since this interchange result does not consider feasibility, Proposition 2 can have application to problems with other constraint sets.

PROPOSITION 2. *Let $\sigma = \pi_1 i \pi_2 j \pi_3$, $\sigma' = \pi_1 j \pi_2 i \pi_3$. $z(\sigma) \leq z(\sigma')$ if and only if $(p_i - p_j) \sum_{\pi_2} w_k + (w_j - w_i) \sum_{\pi_2} p_k \leq w_i p_j - w_j p_i$.*

Proof. Let $v_t = \sum_{\pi_t} p_k$ and $u_t = \sum_{\pi_t} w_k$. Suppose $z(\sigma) \leq z(\sigma')$. Then

$$\begin{aligned} u_2(v_1 + p_i) + u_2(v_1 + p_i) + w_j(v_1 + v_2 + p_i + p_j) \\ \leq w_j(v_1 + p_j) + u_2(v_1 + p_j) + w_i(v_1 + v_2 + p_j + p_i), \\ u_2(v_1 + p_i) + w_j(v_2 + p_i) \leq u_2(v_1 + p_j) + w_i(v_2 + p_j), \\ u_2(p_i - p_j) + (w_j - w_i)v_2 \leq w_i p_j - w_j p_i. \end{aligned}$$

The converse holds similarly.

A long list of corollaries can be generated from Proposition 2; we highlight only the most important ones. Corollaries 2.1 and 2.2 are originally due to Smith.

COROLLARY 2.1. *If $\pi_2 = \{\phi\}$, then $z(\sigma) \leq z(\sigma')$ if and only if $p_i/w_i \leq p_j/w_j$.*

COROLLARY 2.2. *If σ is feasible and if $p_{\sigma(1)}/w_{\sigma(1)} \leq p_{\sigma(2)}/w_{\sigma(2)} \leq \dots \leq p_{\sigma(n)}/w_{\sigma(n)}$, then σ is optimal for (P1).*

Proof. Follows from Corollary 2.1.

The idea for Corollary 2.3 comes from Emmons [1969]. However, our problem is quite different from the one he discusses. This corollary is also a special case of the result found in Rinnooy Kan et al. [1975].

COROLLARY 2.3. *If $d_i \leq d_j$, $p_i \leq p_j$ and $w_i \geq w_j$, then there exists an optimal ordering with $i < j$.*

Proof. Suppose $\sigma' = \pi_1 j \pi_2 i \pi_3$ is an optimal solution of (P1). Since $p_i \leq p_j$, σ is a feasible sequence. The assumptions imply $(p_i - p_j) \sum_{\pi_2} w_k + (w_j - w_i) \sum_{\pi_2} p_k \leq 0$ and $0 \leq w_i p_j - w_j p_i$. Hence, by Proposition 2, $z(\sigma) \leq z(\sigma')$.

As with Corollary 2.3, the next result can establish precedence relations between jobs. This result gives a new method for generating precedence constraints.

PROPOSITION 3. *If $p_i/w_i \geq \max_{1 \leq i \leq r} \{p_i/w_i\}$ and $t > r$, then there exists an optimal ordering with $i < t$ for $i = 1, 2, \dots, r$.*

Proof: Let job i be in position s . If $t < i$ for $1 \leq i \leq r$, $s \geq i + 1$. Define $Q = \{\sigma(1), \sigma(2), \dots, \sigma(s)\}$. Let q be chosen as large as possible so that $p_q/w_q = \max_{k \in Q} p_k/w_k$. Since $t \in Q$, we have $d_q \geq d_i$. Also, because σ is feasible, $d_q \geq \sum_{k \in Q} p_k$. Insert job q at position s and let σ' be the resulting sequence. By Corollary 2.1, $z(\sigma') \leq z(\sigma)$. Now if $t = q$, then we are done. Otherwise, repeat this procedure for the set $Q - \{q\}$ until $t = q$.

To illustrate the use of Proposition 3, assume $n = 6$ and p_i/w_i is 3, 1, 6.2, 3.9, 9, and 4.4 for $i = 1, \dots, 6$ respectively. The precedence relations generated are: $1 < 3$, $2 < 3$; $1 < 4$, $2 < 4$; $1 < 5$, $2 < 5$, $3 < 5$, $4 < 5$; $1 < 6$, $2 < 6$.

COROLLARY 3.1. *If $p_1/w_1 \leq p_i/w_i$ for $i = 2, \dots, n$, then $\sigma(1) = 1$.*

Proof. By Proposition 3, $1 < 2$, $1 < 3$, $\dots$, $1 < n$.

COROLLARY 3.2. *If $p_t/w_t \geq \max_{1 \leq i \leq r} \{p_i/w_i\}$ and $\sum_{k=1}^n p_k \leq d_t$, then there exists an optimal ordering with $i < t$ for $i = 1, 2, \dots, r$ and $i \neq t$.*

When $p_t/w_t = \max_{i=1, \dots, n} \{p_i/w_i\}$ and $\sum_{k=1}^n p_k \leq d_t$, Corollary 3.2 implies that for some optimal solution, job t will appear in the last position. If $p_t/w_t < \max_i \{p_i/w_i\}$, the next result also gives conditions under which t can be put at the end.

PROPOSITION 4. *Suppose $p_t/w_t < \max_i \{p_i/w_i\}$. Let $p_t/w_t = \max_i \{p_i/w_i \mid \sum_{k=1}^n p_k \leq d_i\}$ and let $s = \max\{i \mid p_i/w_i > p_t/w_t\}$. If $\sum_{k=1}^j p_k + p_t > d_j$ for some $j = s, s + 1, \dots, t - 1$, then for some optimal solution σ^* , $\sigma^*(n) = t$.*

Proof. Since $\sum_{k=1}^r p_k + p_t > d_t$ for some $r \in \{s, \dots, t - 1\}$, by Corollary 1.1, $i < t$ for $i = 1, 2, \dots, r$. For each $i = r + 1, \dots, n$, $p_t/w_t \geq p_i/w_i$. Therefore, by Corollary 2.1, t can be put in the last position.

3. A LOWER BOUND

In general, a tight lower bound is needed if a branch and bound method is to be effective. The bound developed by Potts and Van Wassenhove uses a heuristic by Smith to find a feasible ordering of jobs. Potts and Van Wassenhove use a Lagrangian relaxation of (P1) and set the λ 's so that Smith's ordering is optimal. The value PVWBD, of this solution, is a lower bound for (P1). We refer to the method of finding PVWBD as Procedure PVW, and refer to the upper bound found by Smith's heuristic as SMUB.

We now outline an $O(n \log n)$ procedure that produces the lower bound LBD for (P1).

Procedure LB

Step 0. $P = \sum_{i=1}^n p_i$, $\text{CBRK} = 0$, $E = \{i \mid d_i \geq P\}$, $\bar{E} = \{i \mid d_i < P\}$.

Step 1. If $E = \{\phi\}$, then $\text{LBD} = \text{Cost of assignment} + \text{CBRK}$. *STOP*.

Otherwise, $p_t/w_t = \max_{j \in E} \{p_j/w_j\}$ and go to 2.

Step 2. Let $T = \{i \in \bar{E} \mid d_i \geq P - p_t\}$.

If there exists an $i \in T$ with $p_i/w_i > p_t/w_t$, then let

$$r = \max\{i \in T \mid p_i/w_i > p_t/w_t\} \quad \text{and go to 3.}$$

Otherwise, assign t as late as possible (to start at time $P - p_t$).

Set

$$E \leftarrow E + T - \{t\}, \quad \bar{E} \leftarrow \bar{E} - T \quad \text{and} \quad P \leftarrow P - p_t.$$

Go to 1.

Step 3. Break job t into two jobs, one with size $P - d_r$ and the other with size $p_t - (P - d_r)$. Assign weights $w_t(P - d_r)/p_t$ and $w_t(p_t - (P - d_r))/p_t$ to the two jobs, respectively. Schedule the first job as late as possible (to start at time d_r). Set

$$E \leftarrow E + \{r\}, \quad \bar{E} \leftarrow \bar{E} - \{r\},$$

$$\text{CBRK} \leftarrow \text{CBRK} + w_t(P - d_r)(p_t - (P - d_r))/p_t.$$

Give the second job the label t .

Set $P \leftarrow d_r$, $t \leftarrow r$ and go to 2.

To illustrate this method, consider the following four-job example.

Job	1	2	3	4
Deadline	5	9	12	14
Processing time	2	3	6	3
Weight	1	1	4	3

The schedule generated by Procedure LB is

Job	4_2	3_3	1	3_2	2	3_1	4_1	
Time	0	1	3	5	6	9	12	14

where job 3 has been broken into three pieces and job 4 into two pieces. The size and weights of these pieces are

Job	3 ₁	3 ₂	3 ₃	4 ₁	4 ₂
p_i	3	1	2	2	1
w_i	2	2/3	4/3	2	1

The resulting lower bound is

$$\begin{aligned} \text{LBD} &= 1 \cdot 1 + \frac{4}{3} \cdot 3 + 1 \cdot 5 + \frac{2}{3} \cdot 6 + 1 \cdot 9 + 2 \cdot 12 + 2 \cdot 14 \\ &\quad + (4 \cdot 3 \cdot 3)/6 + (2 \cdot 1 \cdot 2)/3 + (3 \cdot 2 \cdot 1)/3 \\ &= 84\frac{1}{3} \end{aligned}$$

where the last three terms of LBD come from CBRK, an additional cost associated with dividing a job. The optimal solution is $\sigma^* = (1, 2, 3, 4)$ with a cost of 93.

To show that Procedure LB is $O(n \log n)$, we note that only when $\bar{E} \neq \{\phi\}$ can a job be broken into two jobs (see Step 2). When this situation occurs, the procedure eliminates one element from $\bar{E}$. Thus, Procedure LB generates at most n additional jobs. Since the job with the largest p/w ratio is never split, this number is actually $n - 1$. The work to find and specify these jobs is $O(n)$. Since the effort to order at most $2n - 1$ jobs is $O(n \log n)$, $O(n \log n)$ is the effort required to perform Procedure LB.

When a job is broken in Procedure LB, each new job retains the same p/w ratio as the original job. Jobs are assigned “greedily” because the feasible job with the largest p/w ratio is assigned to the last free position. This procedure is identical to the assignment procedure used in Smith’s heuristic. $\text{LBD} \leq \text{SMUB}$ because the pieces of a job in Procedure LB do not have to be assigned contiguously. Since we do not want extra reductions in cost to arise because of the creation of additional jobs, we have added the constant $w_t(P - d_r)(p_t - (P - d_r))/p_t$ to LBD. The creation of additional jobs is necessary since simple job splitting only increases the solution cost.

PROPOSITION 5. $\text{LBD} \leq z(\sigma^*)$ where σ^* is an optimal ordering for (P1).

Proof. Let (P2) be the scheduling problem generated by Procedure LB, with the additional jobs and the new processing times and weights.

Assume a job t of (P1) is broken into k pieces with sizes $p_{t_1}, p_{t_2}, \dots, p_{t_k}$ where $\sum_{j=1}^k p_{t_j} = p_t$ and $p_t/w_t = p_{t_j}/w_{t_j}$ for $j = 1, \dots, k$. If job t begins at time T in an optimal ordering σ^* , the cost of this job in the objective function of (P1), $z(\sigma^*)$, is $w_t(T + p_t)$. Suppose, in (P2), job p_{t_k} also starts at T and $p_{t_k}, \dots, p_{t_1}$ are placed contiguously. The additional cost for these k jobs in LBD is ΔLBD_t where

$$\begin{aligned} \Delta\text{LBD}_t &= \text{completion cost} + \Delta\text{CBRK} \\ &= w_{t_k}(T + p_{t_k}) + w_{t_{k-1}}(T + p_{t_k} + p_{t_{k-1}}) + \dots + \\ &\quad w_{t_1}(T + p_{t_k} + \dots + p_{t_1}) + w_{t_1}p_{t_1}(p_{t_k} + \dots + p_{t_2})/p_{t_2} \\ &\quad + w_{t_1}p_{t_2}(p_{t_k} + \dots + p_{t_3})/p_{t_3} + \dots + w_{t_{k-2}}p_{t_{k-1}}p_{t_k}/p_{t_{k-2}}. \end{aligned}$$

Since $w_i/p_t = w_i/p_{t_j}$ for $j = 1, \dots, k$ and $p_t = \sum_{j=1}^k p_{t_j}$, we have $w_t = \sum_{j=1}^k w_{t_j}$. Thus,

$$\begin{aligned} \Delta \text{LBD}_t &= w_t T + \sum_{j=1}^k w_{t_j} \sum_{i=j}^k p_{t_i} + (w_t/p_t)(p_{t_1}(p_{t_k} + \dots + p_{t_2}) \\ &\quad + p_{t_2}(p_{t_k} + \dots + p_{t_3}) + \dots + p_{t_{k-1}}p_{t_k}) \\ &= w_t T + \sum_{j=1}^k w_{t_j} \sum_{i=j}^k p_{t_i} + (w_t/p_t)[p_{t_2}p_{t_1} + p_{t_3}(p_{t_1} + p_{t_2}) \\ &\quad + \dots + p_{t_k}(p_{t_1} + \dots + p_{t_{k-1}})] \\ &= w_t T + \sum_{j=1}^k w_{t_j} \sum_{i=j}^k p_{t_i} + \sum_{j=2}^k w_{t_j} \sum_{i=1}^{j-1} p_{t_i} = w_t T + w_t p_t. \end{aligned}$$

By repeating this analysis for each job $t = 1, \dots, n$, we have a feasible solution to (P2) that is generated by breaking the jobs of σ^* while leaving their positions intact. This solution has the same value as $z(\sigma^*)$.

Let γ be the ordering found by Procedure LB for (P2). We show that γ is optimal by breaking the jobs into pieces. Each job p_i of (P2) is broken into p_i unit size pieces, each with a weight p_i/w_i and a deadline d_i . Smith's ordering for this new problem has each piece contiguously placed and has the set of these pieces occupying the same time slots as p_i does in σ . Potts and Van Wassenhove showed that when all jobs have unit size, then Smith's ordering is optimal. From the previous discussion concerning the breaking of jobs, γ is optimal for (P2). Thus, $\text{LBD} \leq z(\sigma^*)$.

We now show that our bound dominates the lower bound of Potts and Van Wassenhove, as well as the one of Bansal.

PROPOSITION 6. *For any given problem (P1), $\text{LBD} \geq \text{PVWBD}$.*

Proof. Let r be the first job that is broken by Procedure LB (the one in the highest position). It is possible to show that all jobs assigned in positions after job r will have identical assignments in Smith's heuristic and Procedure PVW.

Suppose that, by Procedure LB, the first time a piece of r begins processing is at time T_1 , and the last time it starts is T_2 . We can assume that, if p_i/w_i is not unique, in Step 1 of Procedure LB, then a broken job with a piece assigned to the highest position will be used to break ties. Thus, every job i assigned between T_1 and T_2 has $p_i/w_i > p_r/w_r$. Call this set of jobs G . Since each job assigned to a position before T_1 has either $d_j \leq T_1$ or $p_j/w_j \leq p_r/w_r$, all of the pieces of the jobs in G will be assigned between T_1 and T_2 . By similar reasoning, it is possible to see that Smith's heuristic also assigns all of the elements of G between T_1 and T_2 .

Define C_i as the completion time of job i in Smith's heuristic. If for some $i \in G$, $d_i - C_i = 0$, then some job $j \in G$ with $d_j > d_i$ must be positioned earlier by Smith's heuristic. (In the parlance of Potts and Van Wassenhove, this conclusion implies that the elements of G are in the same block.) If this were not true, then the remainder of job r would

come after job i and i would not belong to G . Since $p_r/w_r < p_i/w_i$ for all $i \in G$,

$$\begin{aligned} \text{PVWBD} &= \text{SMUB} - \sum_{i=1}^n \lambda_i (d_i - C_i) \\ &= \text{SMUB} - \sum_{i \in G} [(p_i w_r / p_r) - w_i] [d_i - C_i] - \sum_{i \notin G} \lambda_i (d_i - C_i). \end{aligned}$$

To compare LBD and PVWBD, we will rearrange Smith's ordering until it corresponds to the one generated by Procedure LB. First, move the job $t \in G$ with the largest p/w ratio. Suppose job t is s_t units of time further back in Procedure LB's ordering. Since job t has size p_t , a total combination of items with size s_t must move p_t units forward. Since each item i that goes forward has $p_i/w_i > p_r/w_r$, the savings from this change is less than or equal to $p_t(s_t w_r / p_r) - s_t w_t$. By repeating this process, using jobs with successively smaller p/w ratios, we have $\text{LBD} \geq \text{SMUB} - \sum_{i \in G} [(p_i w_r / p_r) - w_i] s_i - \text{cost of changing jobs } \notin G$. Since job $i \in G$ can move back at most $d_i - C_i$ units, $s_i \leq d_i - C_i$. (In practice, only part of a job may move back or the job actually be pushed forward.) Thus, $\text{LBD} \geq \text{SMUB} - \sum_{i \in G} [(p_i w_r / p_r) - w_i] [d_i - C_i] - \text{cost of changing jobs } \notin G$.

By repeating this analysis for the jobs completed in time $\leq T_1$, and iterating until all earlier jobs have been accounted for, we obtain the desired result.

Bansal also proposed a lower bound BBD. For this bound, the jobs were assigned in increasing p/w ratio order (see Corollary 2.2). While Potts and Van Wassenhove showed that PVWBD is computationally superior to BBD, they found problems for which $\text{BBD} > \text{PVWBD}$. Our bound dominates BBD.

PROPOSITION 7. *For any given problem (P1), $\text{LBD} \geq \text{BBD}$.*

Proof. From the proof of Proposition 5, breaking up jobs, reassigning the weights, and adding CBRK as in Procedure LB does not change the value of BBD (only rearranging will do that). Bansal's lower bound solution for (P2) is optimal if there are no deadline constraints. Since deadline constraints must be satisfied for LBD, $\text{LBD} \geq \text{BBD}$.

4. A BRANCH AND BOUND ALGORITHM

We have used a Branch and Bound algorithm to solve (P1). The use of such a method is justified since our problem is NP-hard (see Lenstra et al. [1977]).

We outline our method as follows:

- Step 0.* Fix the position of as many jobs as possible (jobs are set to beginning and end positions). Calculate LBD for this initial node.
- Step 1.* Find the node with the smallest lower bound. If all jobs at this

30	0.25	1.1	9	10	18.9	4.9	68	18	92.1	97.5	4.2
		1.4	10	10	24.7	6.5	86	26	94.4	98.3	3.4
		1.7	9	10	18.3	8.8	58	30	97.2	98.7	2.8
		2.0	10	10	17.4	6.1	56	20	97.7	99.0	2.1
	0.50	1.1	10	10	10.6	3.7	29	14	95.4	98.5	3.0
		1.4	10	10	8.6	3.3	25	11	99.1	99.3	2.2
		1.7	10	10	7.4	3.8	20	13	99.5	99.6	2.0
		2.0	10	10	6.6	2.9	18	11	99.5	99.5	1.8
	0.00	1.1	2	9	—	46.5	—	124	95.4	98.6	5.7
		1.4	1	6	—	107.5	—	192	99.1	99.4	0.5
		1.7	2	7	—	77.6	—	192	97.9	98.9	1.2
		2.0	1	7	—	78.5	—	259	97.0	99.2	1.5
	0.25	1.1	6	10	104.6	27.9	261	70	95.9	99.8	6.2
		1.4	4	9	—	31.3	—	83	98.5	99.3	4.0
		1.7	3	7	—	33.9	—	94	99.2	99.5	0.9
		2.0	3	8	—	39.2	—	94	99.6	99.6	2.0
	0.50	1.1	10	10	59.2	21.5	130	48	98.2	99.2	5.0
		1.4	8	10	50.4	20.4	108	44	99.4	99.6	3.2
		1.7	9	10	44.2	17.5	95	37	99.6	99.7	2.5
		2.0	10	10	46.7	20.6	96	45	99.7	99.7	2.3

node are fixed, then STOP. Otherwise, find the set of unscheduled jobs E than can be placed in the last available position.

Step 2. Construct a branch for each element of E . For each new node, fix the corresponding job at the last free position.

Step 3. For each new node fix as many jobs as possible and find LBD. Go to 1.

Corollaries 1.2 and 3.2 and Proposition 4 are used in Steps 0 and 3 to fix jobs at the end positions. Corollaries 3.1 and 1.1 are used to assign jobs to the front.

In Step 1, E is the set of jobs that can be placed in k , the last free position, without exceeding their deadlines. Some elements of E are eliminated using Proposition 4 and Corollaries 2.3 and 2.1. If for r , $t \in E$, $p_r/w_r < p_t/w_t$ and r is fixed at position k , then by Proposition 4 job t must precede $s = \max\{i \notin E \mid p_i/w_i > p_t/w_t\}$. Further, each $t' \in E$ where $p_r/w_r < p_{t'}/w_{t'}$ must precede some $s' = \max\{i \notin E \mid p_i/w_i > p_{t'}/w_{t'}\}$. If these precedence relations eliminate all feasible solutions (see Proposition 1), then for this node r cannot be placed at k . From Corollary 2.3, if $p_r < p_j$ and $w_r > w_j$ for some $j \in E$, again $\sigma(k) \neq r$. Last, suppose position $k + 1$ is occupied by job b . If $p_b/w_b < p_r/w_r$ and it is feasible to place r in position $k + 1$, Corollary 2.1 shows that placing job r in position k is not optimal.

When calculating LBD in Step 3, we use the additional restrictions provided by Proposition 4. If $r \in E$ is selected for position k and $p_r/w_r < p_{t'}/w_{t'}$ for $t' \in E$, we can assume for this node that $d_{t'} = d_{s'}$ where $s' = \max\{i \notin E \mid p_i/w_i > p_{t'}/w_{t'}\}$. Since LBD uses deadline restrictions, this approach may tighten the bound.

To compare our method (MP) with the one proposed by Potts and Van Wassenhove (PVW), we coded both procedures in APL and ran them on a DEC 20 computer. The vectors p and w were generated from a discrete uniform distribution over $[1, 100]$ and $[1, 10]$, respectively. To generate d , we selected two numbers α_1 and α_2 with $\alpha_1 \in \{0, 0.25, 0.5\}$ and $\alpha_2 \in \{1.1, 1.4, 1.7, 2\}$. Then, we generated a set of n d_j 's from a discrete uniform distribution over the integer portion of $\{\alpha_1 \sum_{i=1}^n p_i, \alpha_2 \sum_{i=1}^n p_i\}$. We continued to generate new sets of d_j 's until we found one that produced a feasible solution (see Proposition 1). For each pair of α_1 and α_2 and problem size ($n \in \{10, 20, 30\}$), we ran 10 problems. Table I presents the results. A maximum of 120 seconds of CPU time was allowed for each problem. When five or more problems in a given category took over 120 seconds, the median was replaced by a "—."

From Table I we can see that (MP) is faster and generated fewer nodes than (PVW). This performance is due in part to the reductions incorporated into Step 3 of (MP). In particular, the use of Proposition 4 eliminates many nodes and keeps the tree thin. Unlike (MP), prepro-

cessing is carried out in PVW to create precedence relations and to tighten the lower bound. Some of this preprocessing is automatically implemented in (MP) with little computational effort.

Table I also compares the bounds LBD and PVWBD. In Section 3, we showed that LBD is a tighter bound than PVWBD. Our computational results, which are given for the root node, show the same results. This difference is most noticeable when many of the d 's are restrictions (α_2 is small). Also, PVWBD appears to take slightly more computational effort than LBD. The mean CPU times in seconds for PVWBD are 0.175, 0.358 and 0.521 for $n = 10, 20$ and 30 , respectively. The times for LBD are 0.141, 0.294 and 0.449. In determining PVWBD, the procedure must find a λ for each job. Also, it must do some work to create blocks (see Potts and Van Wassenhove for details). For LBD, while it is theoretically possible to generate an additional $n - 1$ jobs (see Section 3), in practice this outcome rarely occurs. The data indicate that the actual number of additional jobs constructed by Procedure LB is small. This number shrinks as deadlines become less restrictive. Further, the number of additional jobs increases less than linearly with n .

5. CONCLUDING REMARKS

Several of the results in this paper can be easily extended to the problem of minimizing the sum of weighted completion times with release dates. Procedure LB and the bound LBD can be used if they are modified so that jobs are front loaded. LB would then select the currently available job with the smallest p/w ratio. Also, Proposition 4 can be changed so that it holds for the release date problem. This result can be used for tree pruning in a branch and bound algorithm.

ACKNOWLEDGMENT

I would like to thank both referees for outstanding jobs. They have made significant contributions to this work.

REFERENCES

- BANSAL, S. P. 1980. Single Machine Scheduling to Minimize Weighted Sum of Completion Times with Secondary Criterion—a Branch and Bound Approach. *Eur. J. Opnl. Res.* **5**, 177–181.
- BURNS, R. N. 1976. Scheduling to Minimize the Weighted Sum of Completion Times with Secondary Criteria. *Naval Res. Logist. Quart.* **23**, 125–129.
- EMMONS, H. 1969. One-Machine Sequencing to Minimize Certain Functions of Job Tardiness. *Opns. Res.* **17**, 701–715.
- EMMONS, H. 1975. A Note on Scheduling with Dual Criteria. *Naval Res. Logist. Quart.* **22**, 615–616.

- ERSCHLER, J., F. ROUBELLAT AND J. P. VERNHES. 1976. Finding Some Essential Characteristics of the Feasible Solutions for a Scheduling Problem. *Opns. Res.* **24**, 774-783.
- ERSCHLER, J., G. FONTAN, C. MERCE AND F. ROUBELLAT. 1983. A New Dominance Concept in Scheudling n Jobs on a Single Machine with Ready Times and Due Dates. *Opns. Res.* **31**, 114-127.
- HARIRI, A. M. A., AND C. N. POTTS 1983. An Algorithm for Single Machine Sequencing with Release Dates to Minimize Total Weighted Completion Time. *Discrete Appl. Math.* **5**, 99-109.
- LENSTRA, J. K., A. H. G. RINNOOY KAN AND P. BRUCKER. 1977. Complexity of Machine Scheduling Problems. *Ann. Discrete Math.* **1**, 343-362.
- MIYAZAKI S. 1981. One Machine Scheduling with Dual Criteria. *J. Opns. Res. Soc. Jpn.* **24**, 31-50.
- POTTS, C. N., AND L. N. VAN WASSENHOVE. 1983. An Algorithm for Single Machine Sequencing with Deadlines to Minimize Total Weighted Completion Time. *Eur. J. Opnl. Res.* **12**, 379-387.
- RINNOOY KAN, A. H. G. 1976. *Machine Scheduling Problems*, Martinus Nijhoff, The Hague.
- RINNOOY KAN, A. H. G., B. J. LAGEWEG AND J. K. LENSTRA 1975. Minimizing Total Costs in One-Machine Scheduling. *Opns. Res.* **23**, 908-927.
- SHANTHIKUMAR, J. G., AND J. A. BUZACOTT 1982. On the Use of Decomposition Approaches in the Single Machine Scheduling Problem. *J. Opns. Res. Soc. Jpn.* **25**, 29-47.
- SMITH, W. E. 1956. Various Optimizers for Single-Stage Production. *Naval Res. Logist. Quart.* **3**, 59-66.